Quick HandSDK Tutorial (Gaia20_direct_drive)

🤖 **SYSTEM CONFIGURATION**

- **Recommend systems:** Windows 11, Ubuntu 22.04
- **Recommend Languages:** Python 3.12

1. Quick installation

1.1 Conda environment

```bash
conda create --name handsdk python=3.12
conda activate handsdk
```

1.2 Handsdk package installation

1. Cloud Drive Download
2. Click the download button to get "02.HandSDK".

```bash
# Install handsdk package on Linux(x86_64) python version 3.12
pip install handsdk-1.1.1-cp312-cp312-manylinux_2_35_x86_64.whl
```

1.3 USB_CAN permission settings

Our usb-can module will automatically utilize port /dev/ttyACM0 (and /dev/ttyACM1 if dual hands are connected), which requires system permission for further commutication and data transportation.

```bash
sudo chmod 777 /dev/ttyACM0
```

```
sudo chmod 777 /dev/ttyACM1 #if dual hands are connected
```

2. Basic function utilization

2.1 Hand connection and disconnect

```python
from hand import create_hand
from hand.utils.serial_utils import auto_detect_gaia_ports

# Automatically detect serial ports
ports_config = auto_detect_gaia_ports()

# Create a left-hand GaiaHand20 instance.
left_hand = create_hand(hand_type="gaiahand20",
                        hand_side="left",
                        port=ports_config['left'],
                        use_slcan=True)
# Hand connection
left_hand.connect()

# Hand disconnection
left_hand.close()
```

Or you want to connect your right hand

Python

```python
from hand import create_hand
from hand.utils.serial_utils import auto_detect_gaia_ports

# Automatically detect serial ports
ports_config = auto_detect_gaia_ports()

# Create a right-hand GaiaHand20 instance.
right_hand = create_hand(hand_type="gaiahand20",
                         hand_side="right",
                         port=ports_config['right'],
                         use_slcan=True)
# Hand connection
right_hand.connect()

# Hand disconnection
```

```python
right_hand.close() # disconnect and resource release
```

2.2 Hand enable and disable

Python

```python
# Enable all motors
right_hand.enable_all_motors_broadcast(True)
# left_hand.enable_all_motors_broadcast(True)

# Disable all motors
right_hand.enable_all_motors_broadcast(False)
# left_hand.enanble_all_motor_broadcast(False)
```

2.3 Hand joint motion command

2.3.1 Single joint

Python

```python
right_hand.set_joint_angle(
            finger=FingerType.INDEX, # THUMB,INDEX,MIDDLE,RING,LITTLE
            joint=JointType.JOINT_1, # from wrist to tip
            angle=math.radians(45),
            speed=0.5
        )
```

2.3.2 Whole hand

Joint motion command accept both list format and dictionary format, the proper structure will be shown below:

Python

```python
# List format: Contains 16 (one-hand) or 32 (dual-hand) position data (in radians)
# and each finger is arranged from wrist to tip
[
    angle_1, angle_2, angle_3, angle_4,             # Thumb
    angle_1, angle_2, angle_3,                      # Index finger
    ...                                             # In order: middle finger, ring finger, little finger
]

# Dictionary format: more intuitive structured data
```

```
# same as list format, each finger is arranged from wrist to tip
{
    1: {                                        # right hand (1=right hand, 2=left hand)
        1: [angle_1, angle_2, angle_3, angle_4],    # Thumb
        2: [angle_1, angle_2, angle_3],             # Index finger
        ...                                         # 1=thumb, 2=index finger, 3=middle finger, 4=ring finger, 5=little finger
    },
    2: {                                        # left hand (optional, using in dual-hand mode)
        ...
    }
}
```

As a reminder, both formats use **radians** as their input, so here's an example of using *move_joints_pos* to send action

Code block

```
import math
# List format (16 joints)
positions = [
    math.radians(30.0), 0.0, math.radians(27.0), math.radians(43.0),  # Thumb
    0.0, math.radians(43.0), math.radians(56.0),                      # Index finger
    0.0, math.radians(33.0), math.radians(25.0),                      # Middle finger
    0.0, math.radians(28.0), math.radians(18.0),                      # Ring finger
    0.0, math.radians(19.0), math.radians(26.0)                       # Little finger
]

right_hand.move_joints_pos(positions, speed=1, use_broadcast=True)

# Dictionary format
positions_dict = {
     1: {
        1: [math.radians(30.0), 0.0, math.radians(10.0), math.radians(10.0)],  # Thumb
        2: [0.0, math.radians(10.0), math.radians(10.0)],                      # Index finger
        3: [0.0, math.radians(13.0), math.radians(15.0)],                      # Middle finger
```

```
        4: [0.0, math.radians(18.0), math.radians(10.0)],
    # Ring finger
        5: [0.0, math.radians(19.0), math.radians(10.0)]
    # Little finger
    }
}

right_hand.move_joints_pos(positions_dict, speed=1, use_broadcast=True)
```

2.4 Hand state obtain command

Joint states will be organized in **List format**, and all the joint angles will be presented in **radians**

Python

```
# use hand.get_joint_positions() to get joint state
left_hand = left_hand.get_joint_positions()
right_hand = right_hand.get_joint_positions()
```

Here's an example of joint state utilaztion

Python

```
# Get current positions
current_positions = hand.get_joint_positions()
print(f"Current positions: {current_positions}")
```

And you will get something like:

Bash

```
Current positions:
[-0.003408846195301425, 0.473693267299086, 0.7499461629663134,
-0.001090830782496456,
0.7528095937703667, 0.975884488790892, 0.000409061543436171,
0.5775948993318735, 0.43837762071576325, 0.001090830782496456,
0.4901920828843449, 0.31279572688085877, 0.000136353847812057,
0.33011266555299, 0.45705809786601503, 0.3502930350291744]
```

2.5 Hand reset home

Python

```python
# go back to zero position
hand.hand_zero()
```

3. Example

Python

```python
import time
import math
from hand import create_hand, FingerType, JointType
from hand.utils.serial_utils import auto_detect_gaia_ports
from hand.core import set_log_level
from hand import FingerType, JointType

set_log_level('WARNING') # only allow warnings and errors log

def main():
    # 2.1 Hand connection
    ports = auto_detect_gaia_ports()
    hand = create_hand(
        hand_type="gaiahand20",
        hand_side="right",
        port=ports["right"],
        use_slcan=True,
    )
    hand.connect()

    # 2.2 Enable all motors
    hand.enable_all_motors_broadcast(True)
    time.sleep(1)

    # 2.4 Get joint state
    print("current:", hand.get_joint_positions())
    time.sleep(2)

    # 2.3.1 Single joint
    hand.set_joint_angle(
        finger=FingerType.INDEX,
        joint=JointType.JOINT_1,
        angle=math.radians(-30),
        speed=0.5,
    )
    time.sleep(3)

    # 2.3.2 Whole hand — list format (16 joints, radians)
    positions = [
```

```
        math.radians(30.0), 0.0, math.radians(27.0), math.radians(43.0),  # Thumb
        0.0, math.radians(43.0), math.radians(56.0),                      # Index
        0.0, math.radians(33.0), math.radians(25.0),                      # Middle
        0.0, math.radians(28.0), math.radians(18.0),                      # Ring
        0.0, math.radians(19.0), math.radians(26.0),                      # Little
    ]
    hand.move_joints_pos(positions, speed=1, use_broadcast=True)
    time.sleep(3)

    # 2.3.2 Whole hand – dict format
    positions_dict = {
        1: {
            1: [math.radians(30.0), 0.0, math.radians(10.0), math.radians(10.0)],
            2: [0.0, math.radians(10.0), math.radians(10.0)],
            3: [0.0, math.radians(13.0), math.radians(15.0)],
            4: [0.0, math.radians(18.0), math.radians(10.0)],
            5: [0.0, math.radians(19.0), math.radians(10.0)],
        }
    }
    hand.move_joints_pos(positions_dict, speed=1, use_broadcast=True)

    # 2.4 Get joint state
    print("after move:", hand.get_joint_positions())
    time.sleep(2)

    hand.hand_zero()
    time.sleep(3)
    # 2.2 Disable all motors
    hand.enable_all_motors_broadcast(False)

    hand.close()

if __name__ == "__main__":
    main()
```

4. More Info

HandSDK Documentary: HandSDK Development Guidelines (Python Version)